

U2T1_G4 preguntas

Pregunta 1: *En el CRUD de Estudiante, ¿por qué el patrón Composite permite tratar de manera uniforme a un estudiante individual y a un grupo de estudiantes? Explique usando las clases ComponenteAcademico, Estudiante y GrupoEstudiantes.*

Respuesta:

Considero que el patrón composite permite tratar de manera uniforme a un estudiante individual y a un grupo de estudiantes porque el patrón composite como su definición indica permite representar jerarquías de parte-todo, en este ejemplo de CRUD Estudiante las jerarquías sería estudiante individual y grupo de estudiantes.

En el código realizado la clase Estudiante y GrupoEstudiantes comparten la misma abstracción: ComponenteAcademico.

En la implementación realizada la clase Estudiante esta representando la hoja del patrón, es decir el objeto individual que no contiene otros elementos, por otra parte nuestra clase GrupoEstudiantes representan el compuesto, ya que almacena una lista de objetos de tipo ComponenteAcademico.

Gracias a la estructura que se definió, tanto un estudiante como un grupo de estudiantes pueden ejecutar el método mostrarInformacion(). De esta manera nuestro ejemplo de CRUD de Estudiante mantiene una estructura jerárquica clara y flexible, donde los objetos simples y compuestos se manejan bajo una misma interfaz.

Pregunta 2: *¿Qué problema resuelve el patrón Proxy cuando se coloca entre el controlador y el repositorio real del CRUD de Estudiante? Argumente considerando validación de datos, control de acceso y protección de operaciones como crear, actualizar o eliminar.*

Respuesta:

El patrón Proxy resuelve el problema del control y de la validación de datos dentro del repositorio del almacenamiento, así evita que se realicen operaciones incorrectas directamente en la colección real.

Al actuar como un intermediario directo entre el controlador “EstudianteController” y el repositorio real “EstudianteRepositoryReal”, el objeto “EstudianteRepositoryProxy” intercepta todas las solicitudes críticas del CRUD.

Considerando la validación de datos; ejecuta filtros preventivos como “validarEstudiante” para asegurar que la información del alumno cumpla con las reglas de negocio antes de ser procesada.

Considerando el control de acceso y protección de operaciones; evalúa la viabilidad de la transacción, impidiendo la duplicidad de registros, evitando IDs repetidos; y

denegando solicitudes de actualización o eliminación que no cumplan con los criterios mínimos de integridad del sistema.

Pregunta 3: *Compare Composite y Proxy en el CRUD de Estudiante. ¿En qué situación elegiría Composite y en qué situación elegiría Proxy? Justifique su respuesta con un ejemplo concreto del sistema.*

Respuesta:

Situación para Proxy: Para optimizar y asegurar un CRUD de estudiantes, se pueden implementar diferentes variantes del patrón Proxy. Por un lado, se puede usar un Proxy Virtual para ahorrar recursos, el cual retrasa la carga o inicialización de datos pesados del estudiante hasta que el usuario realmente los solicite. Por otro lado, un Proxy de Protección actuará como una barrera de seguridad, interceptando cualquier intento de crear, leer, modificar o eliminar registros, garantizando que el usuario tenga los permisos adecuados antes de permitirle el acceso al objeto real.

Situación para Composite: El patrón Composite se utiliza para construir jerarquías de parte-todo y permite que el sistema trate de manera completamente uniforme tanto a los objetos individuales como a las composiciones de estos objetos. En un CRUD de estudiantes, la implementación ideal del patrón Composite sería para gestionar agrupaciones de estudiantes por ejemplo un curso, de la misma forma en la que se gestionan a estudiantes individuales.